```dart
// lib/pages/stock_movements/stock_movements_page.dart
import 'dart:io';
import 'package:flutter/foundation.dart' show kIsWeb;
import 'package:path_provider/path_provider.dart';
import 'package:printing/printing.dart';
import 'package:universal_html/html.dart' as html;
import 'package:puresip_purchasing/utils/stock_movements_pdf_exporter.dart';
import 'package:easy_localization/easy_localization.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/services.dart';
import 'package:puresip_purchasing/debug_helper.dart';
import
'package:puresip_purchasing/pages/stock_movements/services/movement_utils.dart';
import 'package:puresip_purchasing/widgets/app_scaffold.dart';

class StockMovementsPage extends StatefulWidget {
  const StockMovementsPage({super.key});

  @override
  State<StockMovementsPage> createState() => _StockMovementsPageState();
}

class _StockMovementsPageState extends State<StockMovementsPage> {
  // ==================== الفلاتر ====================
  String? selectedCompanyId;
  String? selectedFactoryId;
  String? selectedItemId;
  String? selectedCategory;
  DateTime? startDate;
  DateTime? endDate;
  String sortOrder = 'desc';

  // ==================== البيانات ====================
  List<Map<String, dynamic>> companies = [];
  List<Map<String, dynamic>> factories = [];
  List<Map<String, dynamic>> allItems = [];
  List<Map<String, dynamic>> filteredItems = [];
  Map<String, double> itemStocks = {};
  Map<String, String> itemNames = {};

  // ==================== حالة الصفحة ====================
  bool isLoading = true;
  bool _isArabic = false;
  List<String> userCompanyIds = [];

  // ==================== بيانات الحركات ====================
  List<QueryDocumentSnapshot> _movements = [];
  bool _isLoadingMovements = false;
  String _lastQueryKey = '';

  @override
  void initState() {
```

```
  super.initState();
  _setupInitialDates();
  _loadInitialData();
}

void _setupInitialDates() {
  final now = DateTime.now();
  startDate = DateTime(now.year, now.month, now.day - 30);
  endDate = now;
}

Future<void> _loadInitialData() async {
  setState(() => isLoading = true);
  await _loadUserCompanies();
  await _loadCompanies();
  await _loadFactories();
  await _loadAllItemNames();
  if (selectedFactoryId != null) {
    await _loadItemsForCurrentFactory();
    await _loadInventory();
    await _loadMovements();
  }
  setState(() => isLoading = false);
}

Future<void> _loadUserCompanies() async {
  try {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) return;

    final userDoc = await FirebaseFirestore.instance
        .collection('users')
        .doc(user.uid)
        .get();

    if (userDoc.exists) {
      final data = userDoc.data();
      if (data != null) {
        userCompanyIds = List<String>.from(data['companyIds'] ?? []);
      }
    }
  } catch (e) {
    safeDebugPrint('[ERROR] Loading user companies: $e');
  }
}

Future<void> _loadCompanies() async {
  if (userCompanyIds.isEmpty) return;

  companies = [];
  for (final companyId in userCompanyIds) {
    final doc = await FirebaseFirestore.instance
        .collection('companies')
        .doc(companyId)
```

```
        .get();

    if (doc.exists) {
      final data = doc.data()!;
      companies.add({
        'id': companyId,
        'nameAr': data['nameAr'] ?? companyId,
        'nameEn': data['nameEn'] ?? companyId,
      });
    }
  }

  if (companies.isNotEmpty && selectedCompanyId == null) {
    selectedCompanyId = companies.first['id'] as String;
  }
}

Future<void> _loadFactories() async {
  if (selectedCompanyId == null) return;

  factories = [];
  final snapshot = await FirebaseFirestore.instance
      .collection('factories')
      .where('companyIds', arrayContains: selectedCompanyId)
      .get();

  for (final doc in snapshot.docs) {
    final data = doc.data();
    factories.add({
      'id': doc.id,
      'nameAr': data['nameAr'] ?? doc.id,
      'nameEn': data['nameEn'] ?? doc.id,
    });
  }

  if (factories.isNotEmpty && selectedFactoryId == null) {
    selectedFactoryId = factories.first['id'] as String;
    await _loadItemsForCurrentFactory();
  }
}

Future<void> _loadAllItemNames() async {
  final snapshot = await FirebaseFirestore.instance.collection('items').get();

  for (final doc in snapshot.docs) {
    final data = doc.data();
    final nameAr = data['nameAr'] as String?;
    final nameEn = data['nameEn'] as String?;

    itemNames[doc.id] = _isArabic
        ? (nameAr ?? nameEn ?? doc.id)
        : (nameEn ?? nameAr ?? doc.id);
  }
}
```

```
Future<void> _loadItemsForCurrentFactory() async {
  if (selectedCompanyId == null || selectedFactoryId == null) return;

  try {
    final movementsSnap = await FirebaseFirestore.instance
        .collection('companies')
        .doc(selectedCompanyId)
        .collection('stock_movements')
        .where('factoryId', isEqualTo: selectedFactoryId)
        .get();

    final Set<String> itemIds = {};
    for (final doc in movementsSnap.docs) {
      final itemId = doc.data()['itemId']?.toString();
      if (itemId != null && itemId.isNotEmpty) {
        itemIds.add(itemId);
      }
    }

    final List<Map<String, dynamic>> itemsForFactory = [];
    for (final itemId in itemIds) {
      try {
        final itemDoc = await FirebaseFirestore.instance
            .collection('items')
            .doc(itemId)
            .get();

        if (itemDoc.exists) {
          final data = itemDoc.data()!;
          final itemCategory = data['category']?.toString() ?? 'raw_material';
          itemsForFactory.add({
            'id': itemId,
            'nameAr': data['nameAr'] ?? itemId,
            'nameEn': data['nameEn'] ?? itemId,
            'category': itemCategory,
          });
        }
      } catch (e) {
        safeDebugPrint('[ERROR] Loading item $itemId: $e');
      }
    }

    setState(() {
      allItems = itemsForFactory;
      _filterItemsByCategory();
    });
  } catch (e) {
    safeDebugPrint('[ERROR] Failed to load items for factory: $e');
  }
}

void _filterItemsByCategory() {
  if (selectedCategory == null || selectedCategory == 'all') {
```

```
      setState(() {
        filteredItems = List.from(allItems);
      });
    } else {
      setState(() {
        filteredItems = allItems
            .where((item) => item['category'] == selectedCategory)
            .toList();
      });
    }

    if (filteredItems.isNotEmpty) {
      if (selectedItemId == null ||
          !filteredItems.any((item) => item['id'] == selectedItemId)) {
        setState(() {
          selectedItemId = filteredItems.first['id'] as String;
        });
        _loadMovements();
      }
    } else {
      setState(() {
        selectedItemId = null;
        _movements = [];
      });
    }
  }

  Future<void> _loadInventory() async {
    if (selectedFactoryId == null) return;

    final snapshot = await FirebaseFirestore.instance
        .collection('factories')
        .doc(selectedFactoryId)
        .collection('inventory')
        .get();

    for (final doc in snapshot.docs) {
      final quantity = doc.data()['quantity'];
      itemStocks[doc.id] = (quantity is num) ? quantity.toDouble() : 0.0;
    }
  }

  Future<void> _loadMovements() async {
    if (selectedCompanyId == null || selectedFactoryId == null) return;

    final queryKey =

'${selectedCompanyId}_${selectedFactoryId}_${selectedItemId}_${startDate}_${endDa
te}_$sortOrder';
    if (_lastQueryKey == queryKey && !_isLoadingMovements) return;
    _lastQueryKey = queryKey;

    setState(() => _isLoadingMovements = true);
```

```
  try {
    Query<Map<String, dynamic>> query = FirebaseFirestore.instance
        .collection('companies')
        .doc(selectedCompanyId)
        .collection('stock_movements')
        .where('factoryId', isEqualTo: selectedFactoryId);

    if (selectedItemId != null && selectedItemId!.isNotEmpty) {
      query = query.where('itemId', isEqualTo: selectedItemId);
    }

    if (startDate != null) {
      query = query.where('date',
          isGreaterThanOrEqualTo: Timestamp.fromDate(startDate!));
    }
    if (endDate != null) {
      final endOfDay =
          DateTime(endDate!.year, endDate!.month, endDate!.day, 23, 59, 59);
      query = query.where('date',
          isLessThanOrEqualTo: Timestamp.fromDate(endOfDay));
    }

    query = query.orderBy('date', descending: sortOrder == 'desc');

    final snapshot = await query.get();
    setState(() {
      _movements = snapshot.docs;
    });
  } catch (e) {
    safeDebugPrint('[ERROR] Loading movements: $e');
  } finally {
    if (mounted) {
      setState(() => _isLoadingMovements = false);
    }
  }
}

// ==================== تحسين اختيار التاريخ ====================

void _setDateRange(DateTime newStart, DateTime newEnd) {
  setState(() {
    startDate = newStart;
    endDate = newEnd;
  });
  _loadMovements();
}

// أزرار سريعة للتواريخ
void _setLast7Days() {
  final end = DateTime.now();
  final start = DateTime(end.year, end.month, end.day - 7);
  _setDateRange(start, end);
}
```

```
void _setLast30Days() {
  final end = DateTime.now();
  final start = DateTime(end.year, end.month, end.day - 30);
  _setDateRange(start, end);
}

void _setLastMonth() {
  final end = DateTime.now();
  final start = DateTime(end.year, end.month - 1, end.day);
  _setDateRange(start, end);
}

void _setLast3Months() {
  final end = DateTime.now();
  final start = DateTime(end.year, end.month - 3, end.day);
  _setDateRange(start, end);
}

void _setLast6Months() {
  final end = DateTime.now();
  final start = DateTime(end.year, end.month - 6, end.day);
  _setDateRange(start, end);
}

void _setLastYear() {
  final end = DateTime.now();
  final start = DateTime(end.year - 1, end.month, end.day);
  _setDateRange(start, end);
}

void _setThisMonth() {
  final now = DateTime.now();
  final start = DateTime(now.year, now.month, 1);
  final end = DateTime(now.year, now.month + 1, 0);
  _setDateRange(start, end);
}

void _setThisYear() {
  final now = DateTime.now();
  final start = DateTime(now.year, 1, 1);
  final end = DateTime(now.year, 12, 31);
  _setDateRange(start, end);
}

// ☑ اختيار يوم محدد (من وإلى)
Future<void> _selectStartDate(BuildContext context) async {
  final DateTime? picked = await showDatePicker(
    context: context,
    initialDate: startDate ?? DateTime.now(),
    firstDate: DateTime(2000),
    lastDate: DateTime(2100),
    helpText: 'select_start_date'.tr(),
    cancelText: 'cancel'.tr(),
    confirmText: 'apply'.tr(),
```

```
  );

  if (picked != null) {
    setState(() {
      startDate = picked;
      if (endDate != null && endDate!.isBefore(picked)) {
        endDate = picked;
      }
    });
    _loadMovements();
  }
}

Future<void> _selectEndDate(BuildContext context) async {
  final DateTime? picked = await showDatePicker(
    context: context,
    initialDate: endDate ?? DateTime.now(),
    firstDate: DateTime(2000),
    lastDate: DateTime(2100),
    helpText: 'select_end_date'.tr(),
    cancelText: 'cancel'.tr(),
    confirmText: 'apply'.tr(),
  );

  if (picked != null) {
    setState(() {
      endDate = picked;
      if (startDate != null && startDate!.isAfter(picked)) {
        startDate = picked;
      }
    });
    _loadMovements();
  }
}

// اختيار شهر وسنة محددين (مع إمكانية اختيار يوم)
Future<void> _selectMonthYear(BuildContext context) async {
  final now = DateTime.now();
  DateTime selectedDate = DateTime(
    startDate?.year ?? now.year,
    startDate?.month ?? now.month,
    startDate?.day ?? 1,
  );

  selectedDate = await showDatePicker(
        context: context,
        initialDate: selectedDate,
        firstDate: DateTime(2000),
        lastDate: DateTime(2100),
        helpText: 'select_date'.tr(),
        cancelText: 'cancel'.tr(),
        confirmText: 'apply'.tr(),
      ) ??
      selectedDate;
```

```
  final start = DateTime(selectedDate.year, selectedDate.month, 1);
  final end = DateTime(selectedDate.year, selectedDate.month + 1, 0);
  _setDateRange(start, end);
}

// اختيار سنة محددة
Future<void> _selectYear(BuildContext context) async {
  final now = DateTime.now();
  DateTime selectedYearDate = DateTime(
    startDate?.year ?? now.year,
    1,
    1,
  );

  final DateTime? picked = await showDialog<DateTime>(
    context: context,
    builder: (context) => AlertDialog(
      title: Text('select_year'.tr()),
      content: SizedBox(
        height: 200,
        width: 200,
        child: YearPicker(
          selectedDate: selectedYearDate,
          firstDate: DateTime(2000),
          lastDate: DateTime(2100),
          onChanged: (date) => Navigator.pop(context, date),
        ),
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: Text('إلغاء'.tr()),
        ),
      ],
    ),
  );

  if (picked != null) {
    final start = DateTime(picked.year, 1, 1);
    final end = DateTime(picked.year, 12, 31);
    _setDateRange(start, end);
  }
}

// اختيار نطاق تاريخ مخصص
Future<void> _selectCustomDateRange(BuildContext context) async {
  final DateTimeRange? picked = await showDateRangePicker(
    context: context,
    firstDate: DateTime(2000),
    lastDate: DateTime(2100),
    initialDateRange: DateTimeRange(
      start: startDate ?? DateTime.now().subtract(const Duration(days: 30)),
      end: endDate ?? DateTime.now(),
```

```
      ),
      saveText: 'apply'.tr(),
      cancelText: 'cancel'.tr(),
    );

    if (picked != null) {
      _setDateRange(picked.start, picked.end);
    }
  }

  String _formatDate(DateTime date) {
    return '${date.year}/${date.month.toString().padLeft(2,
'0')}/${date.day.toString().padLeft(2, '0')}';
  }

  // ==================== PDF ====================

  Future<void> _exportToPDF() async {
    if (_movements.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('no_data_to_export'.tr())),
      );
      return;
    }

    setState(() => _isLoadingMovements = true);

    try {
      // ☑ استخدام نفس بيانات الحركات المعروضة على الشاشة
      final pdfData = await _preparePDFData();
      final pdf = await StockMovementsPdfExporter.generateStockMovementsPdf(
        data: pdfData,
        isArabic: _isArabic,
      );

      final Uint8List bytes = await pdf.save();

      if (kIsWeb) {
        final blob = html.Blob([bytes], 'application/pdf');
        final url = html.Url.createObjectUrlFromBlob(blob);
        final anchor = html.document.createElement('a') as html.AnchorElement
          ..href = url
          ..download =
              'stock_movements_${DateTime.now().millisecondsSinceEpoch}.pdf'
          ..style.display = 'none';
        html.document.body?.children.add(anchor);
        anchor.click();
        html.document.body?.children.remove(anchor);
        html.Url.revokeObjectUrl(url);
      } else {
        final dir = await getTemporaryDirectory();
        final file = File(

'${dir.path}/stock_movements_${DateTime.now().millisecondsSinceEpoch}.pdf');
```

```
        await file.writeAsBytes(bytes);
        await Printing.sharePdf(
          bytes: bytes,
          filename:
              'stock_movements_${DateTime.now().millisecondsSinceEpoch}.pdf',
        );
      }

      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('pdf_exported_successfully'.tr())),
        );
      }
    } catch (e) {
      safeDebugPrint('[PDF ERROR] $e');
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('${'pdf_export_error'.tr()}: $e')),
        );
      }
    } finally {
      if (mounted) {
        setState(() => _isLoadingMovements = false);
      }
    }
  }

Future<Map<String, dynamic>> _preparePDFData() async {
  safeDebugPrint('=== PREPARING PDF DATA ===');
  safeDebugPrint('Movements count: ${_movements.length}');

  final Map<String, List<Map<String, dynamic>>> groupedMovements = {};

  for (final doc in _movements) {
    final data = doc.data() as Map<String, dynamic>;
    final itemId = data['itemId']?.toString() ?? '';
    safeDebugPrint('Processing itemId: $itemId');

    final quantity = (data['quantity'] as num?)?.toDouble() ?? 0;
    final type = data['type']?.toString() ?? 'unknown';
    final date = (data['date'] as Timestamp?)?.toDate() ?? DateTime.now();

    final info = MovementUtils.getMovementTypeInfo(type, quantity);

    if (!groupedMovements.containsKey(itemId)) {
      groupedMovements[itemId] = [];
    }

    groupedMovements[itemId]!.add({
      'date': date,
      'in': info['in'],
      'out': info['out'],
      'type_text': info['type_text'],
    });
```

```
  }

  safeDebugPrint('Grouped items count: ${groupedMovements.length}');

  final List<Map<String, dynamic>> itemsData = [];

  for (final entry in groupedMovements.entries) {
    final itemId = entry.key;
    final movements = entry.value;

    // ☑ تأكد من وجود اسم المنتج
    final itemName = itemNames[itemId] ?? 'unknown_product'.tr();
    safeDebugPrint('Item: $itemId -> $itemName');

    final currentStock = itemStocks[itemId] ?? 0;

    // ☑ ابحث عن الفئة في allItems
    final item = allItems.firstWhere(
      (i) => i['id'] == itemId,
      orElse: () => {'category': 'raw_material', 'nameAr': itemName, 'nameEn':
itemName},
    );
    final itemCategory = item['category'] ?? 'raw_material';

    movements.sort((a, b) => (a['date'] as DateTime).compareTo(b['date'] as
DateTime));

    double netMovement = 0;
    for (final m in movements) {
      netMovement += (m['in'] as double) - (m['out'] as double);
    }
    double openingBalance = currentStock - netMovement;

    double runningBalance = openingBalance;
    final List<Map<String, dynamic>> movementsWithBalance = [];

    for (final m in movements) {
      runningBalance = runningBalance + (m['in'] as double) - (m['out'] as
double);
      movementsWithBalance.add({
        ...m,
        'balance': runningBalance,
      });
    }

    itemsData.add({
      'name': itemName,
      'category': itemCategory,
      'openingBalance': openingBalance,
      'closingBalance': runningBalance,
      'currentStock': currentStock,
      'movements': sortOrder == 'desc' ? movementsWithBalance.reversed.toList() :
movementsWithBalance,
    });
```

```
}

// ... باقي الكود كما هو

  String companyName = '';
  String factoryName = '';

  if (selectedCompanyId != null) {
    final company = companies.firstWhere(
      (c) => c['id'] == selectedCompanyId,
      orElse: () => {'nameAr': '', 'nameEn': ''},
    );
    companyName = _isArabic ? company['nameAr'] : company['nameEn'];
  }

  if (selectedFactoryId != null) {
    final factory = factories.firstWhere(
      (f) => f['id'] == selectedFactoryId,
      orElse: () => {'nameAr': '', 'nameEn': ''},
    );
    factoryName = _isArabic ? factory['nameAr'] : factory['nameEn'];
  }

  return {
    'items': itemsData,
    'companyName': companyName,
    'factoryName': factoryName,
    'startDate': startDate,
    'endDate': endDate,
  };
}

// ==================== واجهة المستخدم ====================

@override
void didChangeDependencies() {
  super.didChangeDependencies();
  final isArabicNow = context.locale.languageCode == 'ar';
  if (_isArabic != isArabicNow) {
    _isArabic = isArabicNow;
    setState(() {});
  }
}

@override
Widget build(BuildContext context) {
  _isArabic = context.locale.languageCode == 'ar';

  if (isLoading) {
    return const Scaffold(body: Center(child: CircularProgressIndicator()));
  }

  return AppScaffold(
    title: 'stock_movements_title'.tr(),
```

```
actions: [
  IconButton(
    icon: const Icon(Icons.picture_as_pdf),
    onPressed: _movements.isEmpty ? null : _exportToPDF,
    tooltip: 'export_pdf'.tr(),
  ),
],
body: Column(
  children: [
    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.grey[100],
      child: Column(
        children: [
          // الصف 1: الشركة والمصنع
          Row(
            children: [
              Expanded(
                child: DropdownButtonFormField<String>(
                  initialValue: selectedCompanyId,
                  isExpanded: true,
                  decoration: InputDecoration(
                    labelText: 'company'.tr(),
                    border: const OutlineInputBorder(),
                  ),
                  items: companies.map<DropdownMenuItem<String>>((c) {
                    return DropdownMenuItem<String>(
                      value: c['id'] as String,
                      child: Text(_isArabic
                          ? c['nameAr'] as String
                          : c['nameEn'] as String),
                    );
                  }).toList(),
                  onChanged: (val) async {
                    setState(() {
                      selectedCompanyId = val;
                      selectedFactoryId = null;
                      selectedItemId = null;
                      selectedCategory = null;
                      factories = [];
                      allItems = [];
                      filteredItems = [];
                      _movements = [];
                    });
                    if (val != null) {
                      await _loadFactories();
                    }
                  },
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: DropdownButtonFormField<String>(
                  initialValue: selectedFactoryId,
```

```
          isExpanded: true,
          decoration: InputDecoration(
            labelText: 'factory'.tr(),
            border: const OutlineInputBorder(),
          ),
          items: factories.map<DropdownMenuItem<String>>((f) {
            return DropdownMenuItem<String>(
              value: f['id'] as String,
              child: Text(_isArabic
                  ? f['nameAr'] as String
                  : f['nameEn'] as String),
            );
          }).toList(),
          onChanged: (val) async {
            setState(() {
              selectedFactoryId = val;
              selectedItemId = null;
              selectedCategory = null;
              allItems = [];
              filteredItems = [];
              _movements = [];
            });
            if (val != null) {
              await _loadItemsForCurrentFactory();
              await _loadInventory();
              await _loadMovements();
            }
          },
        ),
      ),
    ],
  ),
  const SizedBox(height: 8),

  // الصف 2: الفئة والمنتج والترتيب
  Row(
    children: [
      Expanded(
        child: DropdownButtonFormField<String>(
          initialValue: selectedCategory,
          isExpanded: true,
          decoration: InputDecoration(
            labelText: 'category'.tr(),
            border: const OutlineInputBorder(),
          ),
          items: const [
            DropdownMenuItem<String>(
                value: null, child: Text('الكل')),
            DropdownMenuItem<String>(
                value: 'raw_material', child: Text('مواد خام')),
            DropdownMenuItem<String>(
                value: 'packaging', child: Text('مواد تعبئة')),
          ],
          onChanged: (val) {
```

```
      setState(() {
        selectedCategory = val;
      });
      _filterItemsByCategory();
    },
  ),
),
const SizedBox(width: 8),
Expanded(
  child: DropdownButtonFormField<String>(
    initialValue: selectedItemId,
    isExpanded: true,
    decoration: InputDecoration(
      labelText: 'product'.tr(),
      border: const OutlineInputBorder(),
    ),
    items: [
      const DropdownMenuItem<String>(
          value: null, child: Text('الكل')),
      ...filteredItems.map<DropdownMenuItem<String>>((i) {
        return DropdownMenuItem<String>(
          value: i['id'] as String,
          child: Text(_isArabic
              ? i['nameAr'] as String
              : i['nameEn'] as String),
        );
      }),
    ],
    onChanged: (val) {
      setState(() {
        selectedItemId = val;
      });
      _loadMovements();
    },
  ),
),
const SizedBox(width: 8),
Expanded(
  child: DropdownButton<String>(
    isExpanded: true,
    value: sortOrder,
    items: const [
      DropdownMenuItem<String>(
          value: 'desc', child: Text('الأحدث أولاً')),
      DropdownMenuItem<String>(
          value: 'asc', child: Text('الأقدم أولاً')),
    ],
    onChanged: (val) {
      setState(() {
        sortOrder = val!;
      });
      _loadMovements();
    },
  ),
```

```
    ),
  ],
),
const SizedBox(height: 8),

// الصف 3: اختيار التاريخ المحسن
Column(
  children: [
    // عرض نطاق التاريخ الحالي مع أزرار منفصلة لاختيار التاريخ
    Row(
      children: [
        Expanded(
          child: OutlinedButton.icon(
            onPressed: () => _selectStartDate(context),
            icon: const Icon(Icons.calendar_today, size: 16),
            label: Text(startDate != null
                ? _formatDate(startDate!)
                : 'start_date'.tr()),
          ),
        ),
        const SizedBox(width: 8),
        Expanded(
          child: OutlinedButton.icon(
            onPressed: () => _selectEndDate(context),
            icon: const Icon(Icons.calendar_today, size: 16),
            label: Text(endDate != null
                ? _formatDate(endDate!)
                : 'end_date'.tr()),
          ),
        ),
        IconButton(
          icon: const Icon(Icons.tune),
          onPressed: () => _showDateOptionsDialog(context),
          tooltip: 'advanced_options'.tr(),
        ),
      ],
    ),
    const SizedBox(height: 8),

    // أزرار سريعة للتواريخ
    SingleChildScrollView(
      scrollDirection: Axis.horizontal,
      child: Wrap(
        spacing: 8,
        runSpacing: 8,
        children: [
          _buildQuickDateButton('last_7_days', _setLast7Days),
          _buildQuickDateButton('last_30_days', _setLast30Days),
          _buildQuickDateButton('last_month', _setLastMonth),
          _buildQuickDateButton(
              'last_3_months', _setLast3Months),
          _buildQuickDateButton(
              'last_6_months', _setLast6Months),
          _buildQuickDateButton('last_year', _setLastYear),
```

```
                        _buildQuickDateButton('this_month', _setThisMonth),
                        _buildQuickDateButton('this_year', _setThisYear),
                      ],
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 8),

              // زر التحديث
              Row(
                children: [
                  Expanded(
                    child: ElevatedButton.icon(
                      onPressed: _isLoadingMovements ? null : _loadMovements,
                      icon: _isLoadingMovements
                          ? const SizedBox(
                              width: 20,
                              height: 20,
                              child:
                                  CircularProgressIndicator(strokeWidth: 2))
                          : const Icon(Icons.refresh),
                      label: Text(_isLoadingMovements
                          ? 'refreshing'.tr()
                          : 'refresh'.tr()),
                    ),
                  ),
                ],
              ),
            ],
          ),
        ),

        // قائمة الحركات
        Expanded(
          child: _isLoadingMovements && _movements.isEmpty
              ? const Center(child: CircularProgressIndicator())
              : _buildMovementsList(),
        ),
      ],
    ),
  );
}

void _showDateOptionsDialog(BuildContext context) {
  showModalBottomSheet(
    context: context,
    builder: (context) => SafeArea(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          ListTile(
            leading: const Icon(Icons.calendar_month),
            title: Text('select_month_year'.tr()),
```

```
            onTap: () {
              Navigator.pop(context);
              _selectMonthYear(context);
            },
          ),
          ListTile(
            leading: const Icon(Icons.calendar_view_month),
            title: Text('select_year'.tr()),
            onTap: () {
              Navigator.pop(context);
              _selectYear(context);
            },
          ),
          ListTile(
            leading: const Icon(Icons.date_range),
            title: Text('select_custom_range'.tr()),
            onTap: () {
              Navigator.pop(context);
              _selectCustomDateRange(context);
            },
          ),
        ],
      ),
    ),
  );
}

Widget _buildQuickDateButton(String labelKey, VoidCallback onPressed) {
  return ElevatedButton(
    onPressed: onPressed,
    style: ElevatedButton.styleFrom(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
      minimumSize: Size.zero,
      tapTargetSize: MaterialTapTargetSize.shrinkWrap,
      backgroundColor: Colors.grey[200],
      foregroundColor: Colors.black87,
    ),
    child: Text(labelKey.tr(), style: const TextStyle(fontSize: 11)),
  );
}

Widget _buildMovementsList() {
  if (_movements.isEmpty && !_isLoadingMovements) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Icon(Icons.inventory_2, size: 64, color: Colors.grey),
          const SizedBox(height: 16),
          Text('no_movements'.tr()),
          const SizedBox(height: 8),
          Text('try_changing_filters'.tr(),
              style: const TextStyle(color: Colors.grey)),
        ],
```

```
    ),
  );
}

final Map<String, List<Map<String, dynamic>>> groupedMovements = {};

for (final doc in _movements) {
  final data = doc.data() as Map<String, dynamic>;
  final itemId = data['itemId']?.toString() ?? '';
  final quantity = (data['quantity'] as num?)?.toDouble() ?? 0;
  final type = data['type']?.toString() ?? 'unknown';
  final date = (data['date'] as Timestamp?)?.toDate() ?? DateTime.now();

  final info = MovementUtils.getMovementTypeInfo(type, quantity);

  if (!groupedMovements.containsKey(itemId)) {
    groupedMovements[itemId] = [];
  }

  groupedMovements[itemId]!.add({
    'date': date,
    'in': info['in'],
    'out': info['out'],
    'type_text': info['type_text'],
  });
}

return RefreshIndicator(
  onRefresh: _loadMovements,
  child: ListView.builder(
    itemCount: groupedMovements.length,
    itemBuilder: (context, index) {
      final itemId = groupedMovements.keys.elementAt(index);
      final movements = groupedMovements[itemId]!;
      final itemName = itemNames[itemId] ?? 'unknown_product'.tr();
      final currentStock = itemStocks[itemId] ?? 0;

      final item = allItems.firstWhere(
        (i) => i['id'] == itemId,
        orElse: () => {'category': 'raw_material'},
      );
      final itemCategory = item['category'] ?? 'raw_material';

      final categoryText = _isArabic
          ? (itemCategory == 'raw_material'
              ? 'مواد خام'
              : 'مواد تعبئة وتغليف')
          : (itemCategory == 'raw_material'
              ? 'Raw Material'
              : 'Packaging Material');

      movements.sort((a, b) =>
          (a['date'] as DateTime).compareTo(b['date'] as DateTime));
```

```
double netMovement = 0;
for (final m in movements) {
  netMovement += (m['in'] as double) - (m['out'] as double);
}
double openingBalance = currentStock - netMovement;

double runningBalance = openingBalance;
final List<Map<String, dynamic>> movementsWithBalance = [];

for (final m in movements) {
  runningBalance =
      runningBalance + (m['in'] as double) - (m['out'] as double);
  movementsWithBalance.add({
    ...m,
    'balance': runningBalance,
  });
}

final displayMovements = sortOrder == 'desc'
    ? movementsWithBalance.reversed.toList()
    : movementsWithBalance;

return Card(
  margin: const EdgeInsets.symmetric(
      horizontal: 8, vertical: 4), // قلل vertical 4 إلى 8 من
  child: Padding(
    padding: const EdgeInsets.all(12),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Container(
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(
            color: Colors.grey[50],
            borderRadius: BorderRadius.circular(8),
          ),
          child: Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Expanded(
                child: Text(
                  '${'product'.tr()}: $itemName ($categoryText)',
                  style: const TextStyle(
                      fontWeight: FontWeight.bold, fontSize: 16),
                ),
              ),
              Row(
                children: [
                  _buildBalanceItem('opening_balance'.tr(),
                      openingBalance, Colors.blue),
                  const SizedBox(width: 16),
                  _buildBalanceItem('closing_balance'.tr(),
                      runningBalance, Colors.green),
                  const SizedBox(width: 16),
```

```
          _buildBalanceItem('current_stock'.tr(),
              currentStock, Colors.orange),
        ],
      ),
    ],
  ),
),
const SizedBox(height: 12),
SizedBox(
  height: 300,
  child: SingleChildScrollView(
    scrollDirection: Axis.horizontal,
    child: DataTable(
      columnSpacing: 12,
      columns: [
        const DataColumn(
            label: SizedBox(width: 50, child: Text('#'))),
        DataColumn(
            label: SizedBox(
                width: 100, child: Text('date'.tr()))),
        DataColumn(
            label: SizedBox(
                width: 120,
                child: Text('movement_type'.tr()))),
        DataColumn(
            label: SizedBox(
                width: 80, child: Text('incoming'.tr()))),
        DataColumn(
            label: SizedBox(
                width: 80, child: Text('outgoing'.tr()))),
        DataColumn(
            label: SizedBox(
                width: 80, child: Text('balance'.tr()))),
      ],
      rows: List.generate(displayMovements.length, (i) {
        final m = displayMovements[i];
        return DataRow(cells: [
          DataCell(SizedBox(
              width: 50,
              child: Text((i + 1).toString(),
                  textAlign: TextAlign.center))),
          DataCell(SizedBox(
              width: 100,
              child: Text(_formatDate(m['date'] as DateTime),
                  textAlign: TextAlign.center))),
          DataCell(SizedBox(
              width: 120,
              child: Text(m['type_text'] as String,
                  textAlign: TextAlign.center))),
          DataCell(SizedBox(
              width: 80,
              child: Text(
                (m['in'] as double) > 0
                    ? (m['in'] as double).toStringAsFixed(2)
```

```
                                        : '-',
                                    textAlign: TextAlign.center,
                                ))),
                            DataCell(SizedBox(
                                width: 80,
                                child: Text(
                                    (m['out'] as double) > 0
                                        ? (m['out'] as double).toStringAsFixed(2)
                                        : '-',
                                    textAlign: TextAlign.center,
                                ))),
                            DataCell(SizedBox(
                                width: 80,
                                child: Text(
                                    (m['balance'] as double).toStringAsFixed(2),
                                    textAlign: TextAlign.center,
                                ))),
                          ]);
                        }),
                      ),
                    ),
                  ),
                ],
              ),
            ),
          );
        },
      ),
    );
  }

  Widget _buildBalanceItem(String label, double value, Color color) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.end,
      children: [
        Text(label, style: const TextStyle(fontSize: 12, color: Colors.grey)),
        const SizedBox(height: 4),
        Text(
          value.toStringAsFixed(2),
          style: TextStyle(
              fontWeight: FontWeight.bold, color: color, fontSize: 14),
        ),
      ],
    );
  }
}
```